

ZZMahjongBot

flowwalker¹

Botzone 最终积分赛 bot 版本排名 17, 分数 1289
Botzone 最后一次模拟赛 bot 版本排名 11, 分数 1410
2026.6.28
<https://github.com/flowwalker/ZZmahjongBot>

Abstract

国标麻将是具有 10^{48} 状态空间、235 维离散动作空间的不完全信息四人博弈。

笔者将通过自己对架构、特征工程、数据增强、以及数据增强后因效率低下而进行的预处理优化的探索实验和分析,来展示自己的思考和创新性,和自己的想法对于麻将 bot 设计的意义。受限于期末周时间紧迫和资金算力不足,笔者探索的大多架构大多是实验性的(而非疯狂调参竞赛性的),重点训了个别时间预算合适的架构,然而不少因为超参调节缺乏时间原因而失败。经历长时间探索,最终得出几个较有前景的架构思想,以及几个能够 botzone 实战不错的完整 bot,详见下解。

1. 引言

国标麻将是国家体育总局 1998 年颁布的竞技麻将规则,包含 81 种番型,和牌门槛 8 番起和。作为四人不完全信息博弈,Bot 系统面临三个核心挑战:(1) 采取怎样的架构和使用如何的特征最有利于 bot 学到胡牌的价值和技巧而非随机的噪音;(2) 如何在有限训练数据下实现充分利用与避免过拟合;(3) 如何在 Botzone 6 秒、单核 CPU 时限内做出最优决策。

不同于大型比赛中动不动就是几十天的训练只为最终结果,我的本次探索力求

(1) 在极有限算力下进行我认为能系统的探索——覆盖特征、模型、数据、训练、部署五个维度的提升措施和思想;

(2) 探寻对于提升 Bot 真正有帮助的关键创新点

(3) 给出实验中遇到的“理想美好,实际骨感”的现象记录与分析

(4) 简明介绍自己花最多时间的 feature 和 model 架构的探索历程

为了创新性展示和完整性展示充分兼顾,本文前段先重点介

¹ 北京大学,北京,中国 2026 春 wlyAI 基班. Correspondence to: flowwalker <flowwalker@github.com>.

绍自己对于麻将 bot 的创新理论点,然后详细介绍自己对 feature 和 model 维度的完整探索历程,其余可详见 code 文件包或者我的 GitHub 仓库。

注:由于第一次在华为云平台训练模型没有经验,很多 acc 和 loss 日志丢失且未记录,具体数据以下大多为个人实验后印象。

2. 关键创新点

2.1. 创新一: SE 通道注意力机制

Squeeze-and-Excitation (SE) 通道注意力是我从纯 CNN 走向“打牌有点感觉”的第一个转折点。

感觉上,在最初的 3 层纯 CNN 上,模型对所有输入通道一视同仁——手牌的 4 个通道与门风的 1 个通道在卷积核眼中没有区别。SE 模块对每个通道做全局平均池化 (GAP),经两层全连接网络学习通道间依赖关系,输出 0-1 权重向量,逐通道缩放原始特征图:

$$\text{SE}(\mathbf{X}) = \mathbf{X} \odot \sigma(\mathbf{W}_2 \cdot \text{Mish}(\mathbf{W}_1 \cdot \text{GAP}(\mathbf{X}))) \quad (1)$$

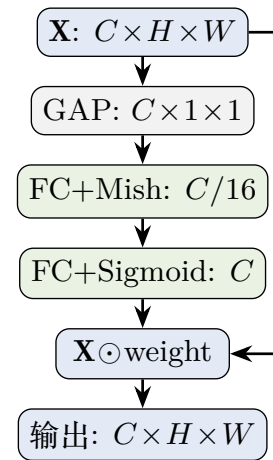


Figure 1. SE 模块。右路跳跃连接表示残差: 原始特征与通道权重逐元素相乘。

以我最终设计的 160 个通道为例,手牌 (4ch)、弃牌历史

(112ch)、对手鸣牌 (21ch) 在不同决策场景下重要性迥异。SE 让模型自行学习注意力分布。引入 SE 使得胡牌水平有明显的提高，而计算开销仅 $O(C^2/r)$ ，可谓十分划算。

2.2. 创新二：特征工程的极致压榨

特征空间经历了 6ch → 16ch → 148ch → 224ch(放弃) → 155ch → 160ch 周期。

飞跃点在 148ch。核心变化是弃牌序列从 4ch 快照扩展为 4 家 × 28 步完整弃牌历史 (112ch)，从后来的对局感觉上，放炮的概率显著降低，推测原因是有了更多的序列记忆当然更容易避免对方的心思。(当然也增加了很多精细化的特征，详见下表。)

同时手牌改为 1-4 张分层编码，鸣牌细化为 16ch。

最终的平衡是 160ch。经历 224ch 膨胀失败 (预计算时间爆炸) 后，最终收敛到 160ch：

Table 1. 160ch 最终特征通道组成

通道	ch	内容
0-3	4	手牌：1-4 张分层编码
4-7	4	明牌：手牌 + 场上可见 (小巧思，揣摩对方心思——增强防御性)
8-28	21	鸣牌：4 家 × 5(吃向/碰/明杠)+ 暗杠
29-32	4	圈风 one-hot
33-36	4	门风 one-hot
37-40	4	牌墙剩余归一化
41-152	112	弃牌历史：4 家 × 28 步打点
153	1	当前待决策牌 (小巧思，不增加计算基础上加快学习，下同)
154	1	牌来源方
155	1	牌墙最后一张标志
156	1	刚杠标志
157-159	3	对手手牌数

这里的 160ch 只包含直接可观测的游戏状态——不做任何语义预处理。将“理解”任务完全交给神经网络。

我最最核心的想法是，由于时间原因，增加到 224ch 计算带来时间开销十分不划算，因此秉承“特征为所见”的思想不断扩增到 160ch，且均确保为无需大处理的特征。

2.3. 创新三 & 四：TTA + 三模型全局软平均投票

这里有一个风险是需要修改 main 函数，因此中间折腾了一段时间，但从结果看还是有一定效果的。

而我也认为这是我最有创新的点子！

TTA (测试时增强) 和多模型投票在部署中协同工作。TTA 对同一局面随机采样 N_{TTA} 个对称变换分别推理后取平均；多模型投票利用 3 个独立训练模型 (同架构、不同随机种子) 的多样性。最终方案为全局软平均——将所有模型 × 所有变换的 logits 直接取平均后 argmax：

$$\text{action} = \arg \max \frac{1}{|\mathcal{M}| \times N_{TTA}} \sum_{m \in \mathcal{M}} \sum_{t=1}^{N_{TTA}} f_m(T_t(x)) \quad (2)$$

软平均保留 logit 分布的完整信息，对模型间置信度差异更鲁棒——一个过于自信的错误模型在硬投票中与正确模型同等权重，在软平均中会被部分稀释。

这里当然也要考虑过其他的例如 rms 和 vote 走法，可见后文提及放弃原因。

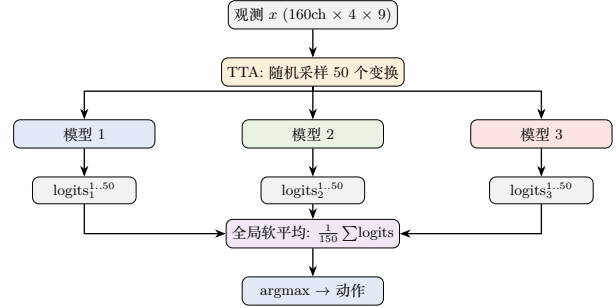


Figure 2. TTA + 多模型投票推理管线。3 × 50 = 150 次前向 (~2.25s)，logits 全局平均后 argmax。

这里的动机是：因为不能确保每一个模型在任何情况下都确保正确，且“训练时模型并未见过足够多的市面”，这样就需要临时增强取平均来减少一下方差，使得模型更加稳健

实测出来的效果是，对于复试对局：连赢四盘的概率增加 (稳健)，但也开始在某些情况出现了过于保守的打牌方式。

2.4. 创新五：模型架构的系统探索

共探索 3 款，11 个架构。本节聚焦两个最重要的创新；完整演进见第5节。

金字塔 CNN-SE (v8)：最终部署模型

金字塔的优势：

- (1) 多尺度特征——深层捕捉细粒度局部模式，浅层保留粗粒度全局结构；
- (2) 参数效率——flat_dim 从 9216 降至 2304，训练速度大幅加快 (这是我最主要的目的)

注：这里的 v 头实际无用，因为不进行 RL 训练，但作为接口保留。

DUAL PATH 双路并行

将时序建模 (BiLSTM) 和空间建模 (CNN) 并行化——共享 CNN 提取底层特征后，分叉为两路独立处理，再通过 Transformer 融合：

下路融合 cnn，最主要的目的是并行，避免 lstm 浪费时间；当然，这也顺便可以视为纯 cnn 外接 lstm 时序建模的扩展。

理论上的直觉是：

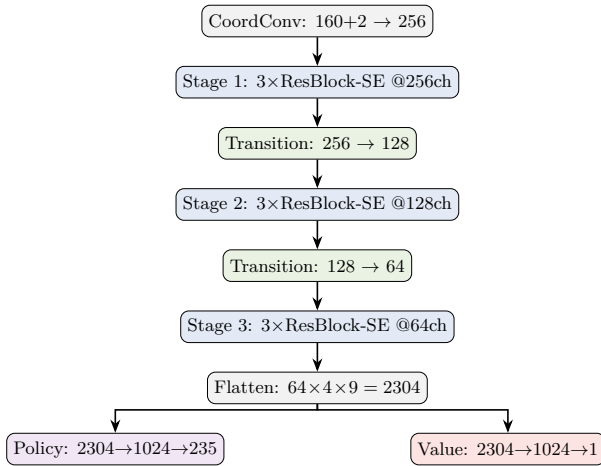


Figure 3. CNN-SE 金字塔 (v8)。三级 256→128→64, 10.4M 参数, flat_dim 仅 2304。

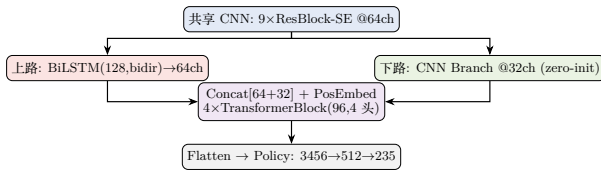


Figure 4. Dual Path (v9 Dual Light, 3.56M)。上路 BiLSTM 时序, 下路 CNN 空间, Transformer 融合。

- 空间感觉用时序建模
- 空间感觉和时间感觉并行
- 注意力自己注意此刻选择何种感觉最为可靠

如果不考虑时间, 这个架构似乎是最为有效的, 与其打牌深感其兵法精妙, 奈何 lstm 在 2x 数据的训练和 lr 调节就十分费力。

最终选 CNN-SE 而非 Dual Path 我认为是一种务实的权衡:

- (1) Dual Path 训练耗时是 CNN-SE 的 2-4 倍 (BiLSTM 无法充分利用 NPU 并行性);
- (2) 同等时间下 CNN-SE 可跑更多 epoch;
- (3) 由于更有时间调参以及 lr 适合, CNN-SE 训练稳定性更好。

但我仍认为 Dual Path 在算力充足时是更有潜力的方向 (见第4.2节)。

2.5. 创新六: 内存预加载

数据供给速度决定能否达到精度上限。为了节省算力带来的金钱花费, 我经历了六代演进:

代	方案	NPU 利用率
v0	逐文件 np.load, 磁盘争抢严重	锯齿状, 常为 0 等待
v1	np.concatenate 扁平化	(没增强) 时间略快
v2	预处理增强 (失败), 磁盘 50GB	一个晚上都加载不完
v3	LazyAug 延迟增强, 缓存优化	可以加载了, 但太慢了
v4	MemPreload 内存预加载	连续 20-40%
v5	分层随机增强 (后来无用)	连续 20-40%

MemPreloadDataset 的核心设计:

1. 全量内存预分配: np.empty 一次性分配三个连续大数组 (int8 压缩 vs float32 爆炸), 避免 np.concatenate 的内存碎片。
2. 纯内存索引: __getitem__ 退化为 int8 解码 + 即时增强变换, 训练全程零磁盘 I/O。这个是非常重要的!
3. LazyAugSampler: 按变换分组 shuffle。

3. “美好理想 → 现实骨感” 的崩溃经验

3.1. “不做选择” 全架构拼接野心的悲哀

理想是: 既然 BiLSTM 擅长时序、Self-Attention 擅长全局关联、CNN 擅长空间、SE 擅长通道——全部拼接在一起! v6 (Gated Fusion,) 和 v7 (Cross-Attention) 通过可学习门控或交叉注意力融合多个分支。

但现实: 完全不可训练。无论怎么调学习率、加 warmup、降 batch size、加 Gradient Clipping——loss 剧烈振荡, val_acc 始终 ~30%+

推测原因是:

- (1) 多分支梯度方向和量级可能造成错配以及巨大的不稳定
- (2) 或许交叉注意力训练初期退化为噪声, 反向传播污染主干网络;

架构复杂度需要与数据和算力匹配。所以只能说优雅可并行、专注的设计远比复杂、全面的设计有优势。

3.2. v 头的麻烦

正常的理想: 我的想法, 或者说正常人都想, SL 预训练后, PPO 自我对弈微调——模型在与自己博弈中超越人类数据。

现实是 v 头太难搞了。

三个失败的尝试:

(1) Value Head 干扰 Policy: SL 只优化 Policy Head, Value Head 随机初始化。首次 RL 更新 value_loss ≈ 12,000——崩。

(2) 算力时间不足: 从头训根本是天方夜谈, 更何况是时间紧迫的期末周。

(3) **SL 阶段 v 头不可训**：纵是 sl 阶段利用稀疏的结果 score 进行奖励，也完全没法让 bot 懂得什么好什么坏，只知道什么结束了，v 头照样爆。

rl 放弃。

3.3. 12x 增强的过拟合

实验 12x 数据增强的时候遇到了如下几个问题：（首先核心是时间翻了 12 倍）

- 学习率未改变直接改成 12x，发现第二个 epoch 就过拟合了
- 调小 lr 重训发现 acc 涨得太慢了，根本没时间收敛
- 又稍微调高 lr，结果又过拟合了。
- 尝试在第一个 epoch 后调小 lr 微调，也过拟合。

推测原因是，全排列 6x 似乎有点让模型矫枉过正，提供过多重复样本而容易导致过拟合。但后来没有时间没有进一步实验了。比赛直接采用 2x。

3.4. 准确率的抛物线怪象

直觉上理论上应该是过拟合后 acc 微微下降，但麻将训练是过拟合后立刻暴跌，很反直觉。

猜测是面对“无能局面”（无论选什么动作都输）时，人类动作本质随机。模型过拟合时开始“记忆”这些随机标签。

3.5. 288x 随机增强是个“灾难”

激动的尝试 288x，因为 288x 包含着我对箭牌和风牌数据增强的良苦创新，课件只提供了 12x 增强的想法

结果一天的金钱和时间离我而去。后来立刻意识到应该先做 2x 估算 288x 时间，一算发现 1epoch 都要上几十天。

于是进行了三种尝试想充分利用 288x：怀着认为利用随机性让他充分见到各种增强的局面。

- (1) 跨 batch 随机 4x——几乎零提升；
- (2) batch 内逐个随机变换——略有下降；
- (3) batch 内逐个随机 5x——val_acc 暴跌至 ~30%。灾难！

推测的原因：增强的核心价值是让模型看到同一局面的不同对称版本。batch 内每个样本使用不同随机变换时，模型看到的是混乱信号——不同变换对同一局面产生矛盾梯度更新。

因此，最终认为增强的“多样性”需要与“一致性”平衡。我采取方案先（固定 2x + 同 batch 同变换 + GroupNorm）再尝试 12x 等等（如果有时间的话，而事实上无）

3.6. 十分之困惑：Botzone 超时与不超时

使用 $N_{TTA} = 50$ ，自行测试两天零超时（有对局记录为证，不可深究-serious-版本 14），但正式计分赛时有超时崩溃扣分，非常让人困惑。

我的推测：计分赛平台同时运行大量对局，CPU 资源被争抢，单核性能下降——所以感觉是 Botzone 缺乏资源隔离机制问题，若不然我的成绩必然能达到一个更高的水平。

3.7. 224ch 专家特征的 Python 瓶颈

理想情况是：将牌效分析、安全度、听牌判断等专家知识编码为额外 76ch 特征 (224ch)，让模型不需要学习已知规律。

但现实：通过 Python 版 MahjongFanCalculator 实时计算番型，单次 `_update_obs()` 耗时约数百毫秒。一局 100-200 次决策需数十秒预处理。490 万样本的预处理时间远超 NPU 训练时间。（当然也可能是因为这部分处理算法后来就没有深入研究）

只能说有点讽刺：添加专家特征是为了“省训练时间”，但 Python 预处理耗时远超模型学习这些规律所需的时间。

3.8. 关于数据增强时均值的选择——主要是 rms 的失败

当时考虑过如下几个方案：

- 采用 vote 走法，选投票最多的走法——但考虑到很大可能每个增强情况想法不同，且动作空间不小，可能都差不多导致失效，但未来可能可以尝试
- rms，即采用平方平均值来替代 vote 软平均，动机是：平方可以增大正确的量级、缩小不合适的走法量级，但是实际效果暴跌——推测是
 - 平方可以增大正确量级也会放大盲目自信的量级因此造成不稳定
 - 平方后概率不归一化，可能造成奇怪现象

4. 最终方案与理想方案

4.1. 最终提交的 Botzone 版本

综合所有约束和实验结果：

组件	选择
特征	160ch，仅直接可观测信息
模型	CNN-SE Pyramid (v8)
增强	2x 分层增强（数字对称 $\mathbb{Z}_2$ ）
数据	int8 压缩，33GB 内存预加载
训练	Adam+CosineAnnealing+LabelSmooth
部署	3 模型 ×50 TTA 软平均投票（测试赛不加更佳）

4.2. 理想中的无限资源训练流程

如果时间、金钱、算力均无约束，

组件	选择
特征	160ch, 仅直接可观测信息 (甚至 224ch 预处理, 关键在时间)
模型	CNN-SE Pyramid-双路 (BiLstm/零权重 cnn)-transformer
增强	288× 全分层增强 + 精细的 lr 调节匹配
数据	int8 压缩, 内存预加载
训练	Adam+CosineAnnealing
部署	k 模型 × n TTA 软平均投票 (多轮测试选最佳方案)

小可能: 利用 sl 好的此模型当老师, 用于快速提升 rl 模型, 随后让 rl 进行自对弈。

5. 探索历程: Feature 与 Model 的演进

5.1. 特征工程演进 (features/)

v1_6ch_minimal.py 一起点: 门风 + 圈风 + 手牌 (4ch 堆叠), 无弃牌/鸣牌/牌墙。(提供的 sample)

v2_16ch_visible.py 一睁开双眼: + 弃牌标记 (4ch) + 鸣牌 (4ch) + 明牌 (1ch) + 牌墙 (1ch)。首个“可用”版本。

v3_148ch_discard_history.py 一质的飞跃: 弃牌序列 4ch → 112ch (4 家 × 28 步完整历史), 手牌分层编码, 鸣牌 16ch。设计洞察: 弃牌顺序和节奏蕴含对手手牌倾向。

v4_224ch_semantic.py 一膨胀顶点: +76ch 语义 + 专家特征。因 Python 预处理瓶颈被放弃 (见第3.7节)。

v5_155ch_aux_state.py 一返璞归真: +7ch 辅助状态 (待决策牌、牌来源、手牌数等)。降低模型推断负担。

v6_160ch_meld21.py 一最终收敛: +5ch MELD 精细化 (吃牌左/中/右方向编码, 暗杠独立)。“直接可观测信息”路线的终稿。

Table 2. 特征工程演进

版本	ch	核心变化
v1	6	手牌 + 风位, sample 基本可用
v2	16	+ 弃牌/鸣牌/牌墙, 首个可用
v3	148	112ch 弃牌历史和更多精细特征, 飞跃
v4	224	+76ch 非常耗时间的专家语义, 被放弃
v5	155	+7ch 辅助状态
v6	160	+5ch 更精细, 最终方案

5.2. 模型架构演进 (models/)

第一阶段: 纯 CNN 探索 (v1-v3)

v1: 3 层 Conv2d, sample. **v2**: 首次引入 **SE+CoordConv+Mish+GroupNorm**, 效果显著提升, SE 自注意力非常有帮助。 **v3**: v2 的 16 块 ResBlock 放大版, 验证了“更大不等于更好”, 所以后期不再一味尝试堆大模型。

第二阶段: 序列建模与膨胀 (v4-v7)

v4_bilstm_sa_148ch.py: CNN+SE 串接 BiLSTM(128,bidir)+SelfAttention(256,8 头)。效果最优——实战和牌和出牌技巧优于同期 CNN-SE。但训练耗时是 CNN-SE 的 3-4 倍, 在数据增强时不够时间调参和训好。

v5_dual_tower_224ch.py: 策略/价值双塔独立。稍微尝试训练发散失败。 **v6_gated_fusion_224ch.py**: 门控融合, 未实验。 **v7_multibranch_cross_attn_224ch.py**: Cross-Attention 融合 (158M), 完全不收敛 (见第3.1节)。

第三阶段: 回归与收敛 (v8)

v8_cnn_se_pyramid_160ch.py: 最终部署模型。16 块平战 256ch → 9 块金字塔 (256 → 128 → 64) (前面提到过的省参数, 加训练速度)

第四阶段: 新型架构探索 (v9-v11)

v9_dual_light_160ch.py: Dual Path 轻量版。水平与 CNN-SE 相当但不够稳定。但因为是 light 可以理解

v10_dual_zerocnn_160ch.py: 消融版, 未实验。 **v11_cnn_transformer_160ch.py**: CNN 金字塔 + Transformer, 未实验。

Table 3. 模型架构演进全景

版本	架构	参数	状态
v1	3 层 CNN, 6ch	0.7M	起点
v2	CNN-SE 4Block	~3M	引入 SE
v3	CNN-SE Large	~29M	大模型探索
v4	BiLSTM+SA	~29M	最优但慢
v5	Dual Tower	~158M	发散
v6	Gated Fusion	~20M	未实验
v7	Cross-Attention	~158M	完全失败
v8	CNN-SE Pyr.	~10.4M	最终部署
v9	Dual Light	相当但不稳	
v10	Dual ZeroCNN	~16.9M	未实验
v11	CNN+Transformer	-	未实验

6. 总结

由于个人强烈的探索欲望, 本人并没有非常执着于暴力堆砌某个数据增强, 疯狂调 lr 等超参数为了拿到前几, 只为了探索什么样的架构和形式能够真正有帮助, 把经验留给真正的未来。

以上部分仅仅详细介绍了最为关键的 feature 和 model 的探索历程, 这也是我花时间思考最多的部分, 其余相对简略, 可以参见 code 文件包或者 GitHub 仓库。

关于预处理的 trick 和数据增强的 trick 可以参见 code 文件夹中对应部分的文件夹, 里面记录了我对于该部分的探索历程。由于部分迁移由 AI 完成, 可能会有些许差错 (小概率, 个人已经花大力气检查过) 但思路毫无问题, 而具体版本则直接参见完全无错的 bot_All_Versions 文件夹即可, 文件命名及架构主要特征。

致谢

感谢北京大学 AI 基础课程提供的平台、Botzone 评测环境和华为云 NPU 算力券的支持。

影响声明

本文展示了极有限算力下构建麻将 Bot 开发工程实践。文中技术方案可推广到其他具有对称性结构的离散动作空间游戏。本文不涉及任何伦理或社会负面影响。所有模型在北大提供的高质量 data.txt 数据集上训练，未收集隐私数据。

References

- Chen, J.-C., Tang, S.-C., and Wu, I.-C. Monte-Carlo simulation for mahjong. *Journal of Information Science and Engineering*, 38:775–790, 2022. doi: 10.6688/JISE.202207_38(4).0005.
- Gao, S., Okuya, F., Kawahara, Y., and Tsuruoka, Y. Building a computer mahjong player via deep convolutional neural networks, 2019.
- Tang, S.-C., Chen, J.-C., and Wu, I.-C. Applying importance sampling to MCTS for mahjong. *IEEE Transactions on Games*, 17(3):743–752, 2025. doi: 10.1109/TG.2025.3535740.
- Yan, X., Li, Y., and Li, S. A fast algorithm for computing the deficiency number of a mahjong hand. *arXiv preprint*, 2021.
- 代君学, 李霞丽, 刘博, and 王昭琦. 国标麻将的多尺度骨干神经网络模型. 重庆理工大学学报 (自然科学), 38(5): 137–144, 2024. 中央民族大学.
- 国家体育总局. 中国麻将竞赛规则 (国标麻将), 1998.
- 王鑫超. 雀圣解说: 一种国标麻将 ai 自动解说方法. Master’s thesis, 北京大学, 2020. 导师: 李文新教授.